

PYTHON

1. Python Indentation

Indentation means the spaces at the beginning of a line.

Unlike languages such as Java, Python uses indentation to identify blocks of code.

Example

```
if age >= 18: print("You are eligible")
```

Here, `print()` belongs to the `if` block because it is indented.

If indentation is incorrect:

```
if age >= 18: print("You are eligible")
```

Python gives an indentation-related error.

What I understood

Indentation in Python is not just for making code look clean. It is part of Python's syntax.

2. Comments

Comments are notes written inside the code for explaining something.

Python ignores comments while executing the program.

```
# Calculate total price price=100 quantity=5 total=price*quantity
```

Comments are useful when the reason behind some code is not immediately obvious.

Types

```
# Single-line comment
```

For longer explanations, Python commonly uses triple-quoted strings, although these are technically string literals rather than comments.

3. Functions

A function is a reusable block of code designed to perform a particular task.

```
def add(a,b):  
    return a+b  
result = add(10,20)  
print(result)
```

Why functions?

Without functions:

```
Code  
Code  
Code  
Code  
Code  
Code
```

With functions:

```
main program  
|  
├─ calculate()  
├─ validate()  
├─ clean()  
└─ save()
```

Functions make code easier to:

- Reuse
 - Test
 - Understand
 - Maintain
-

4. OOP — Object-Oriented Programming

OOP is a programming approach where programs are organized around **objects and classes**.

Basic idea

```
Class
↓
Object
↓
Data + Methods
```

Example:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def display(self):
        print(self.name, self.age)
student1 = Student("Rahul", 20)
student1.display()
```

Important OOP concepts

- Class
- Object
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

What I understood

OOP helps organize larger programs by combining related data and functionality together.

5. File Handling

Python can be used to create, read, modify and write files.

Common modes:

Mode	Meaning
r	Read
w	Write
a	Append
x	Create

Example:

```
withopen("data.txt","r")asfile:content=file.read()print(content)
```

Using `with` is useful because the file is properly closed after the operation.

Why file handling matters

Applications often need to work with:

- CSV files
 - Text files
 - Configuration files
 - Logs
 - JSON files
 - Reports
-

6. Exception Handling

Errors can happen while a program is running.

Python provides `try`, `except`, `else` and `finally` for handling exceptions.

```
try: number = int(input("Enter number: "))
result = 10 / number
except ValueError: print("Please enter a valid number")
except ZeroDivisionError: print("Cannot divide by zero")
```

Basic flow

```
try
↓
Something goes wrong?
↓
Yes → except
↓
Continue safely
```

What I understood

Exception handling prevents the entire program from crashing unexpectedly and allows us to handle known problems properly.

7. NumPy

NumPy is a Python library mainly used for numerical computing.

One of its important features is the `ndarray`, which allows efficient operations on arrays.

```
import numpy as np
numbers = np.array([10, 20, 30, 40])
print(numbers * 2)
```

Output:

```
[20 40 60 80]
```

NumPy is commonly used in:

- Numerical calculations
 - Arrays
 - Matrices
 - Scientific computing
 - Data processing
-

8. Pandas

Pandas is a Python library used mainly for **working with structured data**.

For example, CSV data:

Name	Age	Marks
A	20	80
B	21	75
C	20	90

Can be loaded using:

```
import pandas as pd
data = pd.read_csv("students.csv")
print(data)
```

Important Pandas structures:

Series

One-dimensional data.

DataFrame

Two-dimensional tabular data.

	Name	Age	Marks
0	A	20	80
1	B	21	75
2	C	20	90

Pandas becomes particularly important for the **Parallel CSV Processing Framework** project.

9. Selenium for Web Scraping

Selenium is a browser automation tool.

It can control browsers programmatically.

Basic idea:

```
Python
↓
Selenium
↓
Browser
↓
Website
```

Example:

```
from selenium import webdriver
driver = webdriver.Chrome()
driver.get("https://example.com")
print(driver.title)
driver.quit()
```

Selenium is useful when a website requires browser interaction or dynamically loads content with JavaScript.

Important point

Web scraping should respect the website's terms, robots policies where applicable, rate limits, and applicable laws.

10. Regular Expressions — Regex

Regex is used to search for patterns in text.

Python provides the `re` module.

Example:

```
import re
text = "My email is test@example.com"
pattern = r"\w+@\w+\.\w+"
result = re.search(pattern, text)
print(result.group())
```

Regex can be useful for:

- Email validation
 - Phone number patterns
 - Searching text
 - Extracting information
 - Data cleaning
-

11. Multithreading

Multithreading means having multiple threads executing within a process.

It can be useful for tasks where the program spends significant time **waiting**, such as:

- Network requests
- File operations
- I/O operations

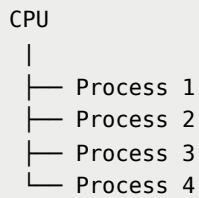
Conceptually:

```
Program
|
├─ Thread 1 → waiting for file/network
├─ Thread 2 → processing I/O
└─ Thread 3 → processing I/O
```

12. Multiprocessing

Multiprocessing uses multiple processes.

Each process has its own Python interpreter and memory space.



It can be useful for **CPU-intensive tasks**.

Examples:

- Heavy calculations
- CPU-intensive data transformation
- Image processing
- Large computational workloads

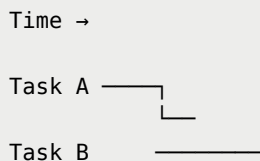
13. Concurrency vs Parallelism

These two terms confused me initially, so I would document them separately.

Concurrency

Multiple tasks are being managed during the same period of time.

They don't necessarily execute at exactly the same instant.



Parallelism

Multiple tasks actually execute at the same time, usually using multiple CPU cores.



Simple difference

Concurrency = dealing with multiple tasks.

Parallelism = executing multiple tasks at the same time.

This distinction becomes important in the CSV processing project.



PART B — SOFTWARE DEVELOPMENT

14. SDLC

SDLC = Software Development Life Cycle

It describes the different stages involved in developing software.

```
graph TD;
    A[Requirements] --> B[Planning];
    B --> C[Design];
    C --> D[Development];
    D --> E[Testing];
    E --> F[Deployment];
    F --> G[Maintenance];
```

Why SDLC?

It gives the team a structured way of developing and maintaining software.

15. Agile

Agile is a software development approach based around:

- Iterative development
- Continuous feedback
- Adaptability
- Frequent delivery
- Collaboration

Instead of trying to build everything at once:

```
Plan → Build everything → Test → Release
```

Agile commonly works more like:

```
Plan
↓
Small feature
↓
Build
↓
Test
↓
Feedback
↓
Improve
↓
Next feature
```

16. Scrum

Scrum is a framework commonly used to organize Agile development.

Common Scrum roles

- Product Owner
- Scrum Master
- Developers

Common events

- Sprint
- Sprint Planning
- Daily Scrum
- Sprint Review
- Sprint Retrospective

Sprint

A Sprint is a fixed period during which the team works on selected work.

17. Version Control

Version control tracks changes made to files/code over time.

Git is a widely used version-control system.

Basic flow:

```
Working Directory
  ↓
git add
  ↓
Staging Area
  ↓
git commit
  ↓
Repository
  ↓
git push
  ↓
GitHub
```

Useful commands:

```
git initgit statusgit add .git commit-m"Initial commit"git pushgit pullgit clone
```

18. Documentation

Documentation explains how a software system works.

It can include:

- Project overview
- Installation instructions
- Architecture
- API documentation
- Configuration
- Usage
- Troubleshooting
- Design decisions

Good documentation should answer:

- | What is it?
 - | Why does it exist?
 - | How do I run it?
 - | How does it work?
 - | What problems can occur?
-

19. Risk Management

Risk management means identifying possible problems before or during a project and deciding how to handle them.

Example:

Risk	Impact	Possible Solution
Data loss	High	Backup

Risk	Impact	Possible Solution
Dependency failure	Medium	Version pinning
Security vulnerability	High	Regular updates
Performance problem	Medium	Benchmarking
Developer unavailable	Medium	Documentation

Basic process:

```
Identify
  ↓
Analyze
  ↓
Prioritize
  ↓
Mitigate
  ↓
Monitor
```

20. Python Coding Standards

Coding standards are guidelines that make code consistent and easier to maintain.

They cover things such as:

- Naming
- Formatting
- Imports
- Functions
- Comments
- Error handling
- File organization

21. PEP 8

PEP 8 is the Python Style Guide.

It provides recommendations for writing readable Python code.

Example:

Instead of:

```
x=10y=20result=x+y
```

Prefer:

```
x=10y=20result=x+y
```

PEP 8 also discusses naming conventions, indentation, imports, whitespace and code layout.

Why I need it

The main purpose is consistency and readability, especially when multiple developers work on the same project.

22. Comments vs Docstrings

These are related but different.

Comment

```
# Calculate total pricetotal=price*quantity
```

Used mainly to explain a line or section.

Docstring

```
defcalculate_total(price,quantity):"""Calculate and return the total price."""returnprice
```

```
*quantity
```

A docstring documents a function, class or module and can be accessed programmatically.

23. Error Handling and Logging

Error handling tells the program **what to do when something goes wrong**.

Logging records useful information about what happened.

```
import logging
logging.basicConfig(level=logging.INFO)
logging.info("Application started")
logging.warning("File is missing")
logging.error("Database connection failed")
```

Difference

```
Exception Handling
  ↓
How the program handles the problem

Logging
  ↓
How we record what happened
```

24. Efficient Code

Efficient code means code that uses resources reasonably while still being readable and maintainable.

Important resources include:

- Execution time
- Memory
- CPU
- Disk I/O

- Network usage

Example:

If a file contains one million records, reading and processing everything inefficiently can consume a lot of memory.

Instead, processing data in chunks can be better:

```
for chunk in pd.read_csv("large.csv", chunksize=10000): process(chunk)
```

This idea is directly relevant to the CSV processing project.

25. Programming Principles

Some useful principles I learned/researched are:

DRY — Don't Repeat Yourself

Avoid unnecessary duplication.

KISS — Keep It Simple

Prefer a simple understandable solution over unnecessary complexity.

SRP — Single Responsibility Principle

A function/class should have a clear responsibility.

Separation of Concerns

Different parts of the application should handle different responsibilities.

For example:

```
Data Loading
  ↓
Validation
  ↓
Cleaning
  ↓
Transformation
```

↓
Aggregation

Each stage has a different responsibility.

26. Unit Testing & Validation

Unit Testing

Testing a small individual unit of code, usually a function.

Example:

```
def add(a, b): return a+b
```

A test can check:

```
assert add(2,3)==5
```

Validation

Validation checks whether input/data meets expected requirements.

Example:

```
Age = 25      → Valid  
Age = -10    → Invalid  
Email correct → Valid  
Email incorrect → Invalid
```

For the CSV project, validation is especially important before cleaning and transformation.

27. Ruff

Ruff is a fast Python linter and code-quality tool.

It can identify many common issues such as:

- Unused imports
- Style issues
- Certain programming mistakes

Example:

```
ruff check .
```

It can be integrated into development workflows so that problems are caught early.

28. Black

Black is a Python code formatter.

Instead of developers manually formatting code, Black applies a consistent formatting style.

For example:

```
black .
```

Ruff vs Black

Ruff	Black
Mainly linting/code quality	Mainly formatting
Finds many code issues	Reformats code
Very fast	Automatic formatter

Modern projects can use both.

29. Final Understanding — Python

After studying these topics, I understood that Python is not only about writing syntax.

A real Python project also requires:

Python Basics

↓

Functions / OOP

↓

File & Data Handling

↓

Error Handling

↓

Testing

↓

Code Quality

↓

Version Control

↓

Documentation

↓

Performance

↓

Maintainability